HomeCam: Android Camera & Viewer App

Nov 9, 2026 · @NITISH KUMAR SHARMA

Overview

HomeCam is one Android app with two modes: **Camera mode** turns an old phone into a CCTV camera, and **Viewer mode** watches it live from another phone. The same APK is installed on both phones; the first screen asks which role this phone plays.

Mode	Runs on	What it does
Camera (caster)	Old phone (e.g. Realme RMX2185)	Captures video, serves it as an MJPEG stream over HTTP, detects motion
Viewer	Your daily phone (e.g. Galaxy S24 FE)	Saves cameras, plays the live stream, takes snapshots, shows status

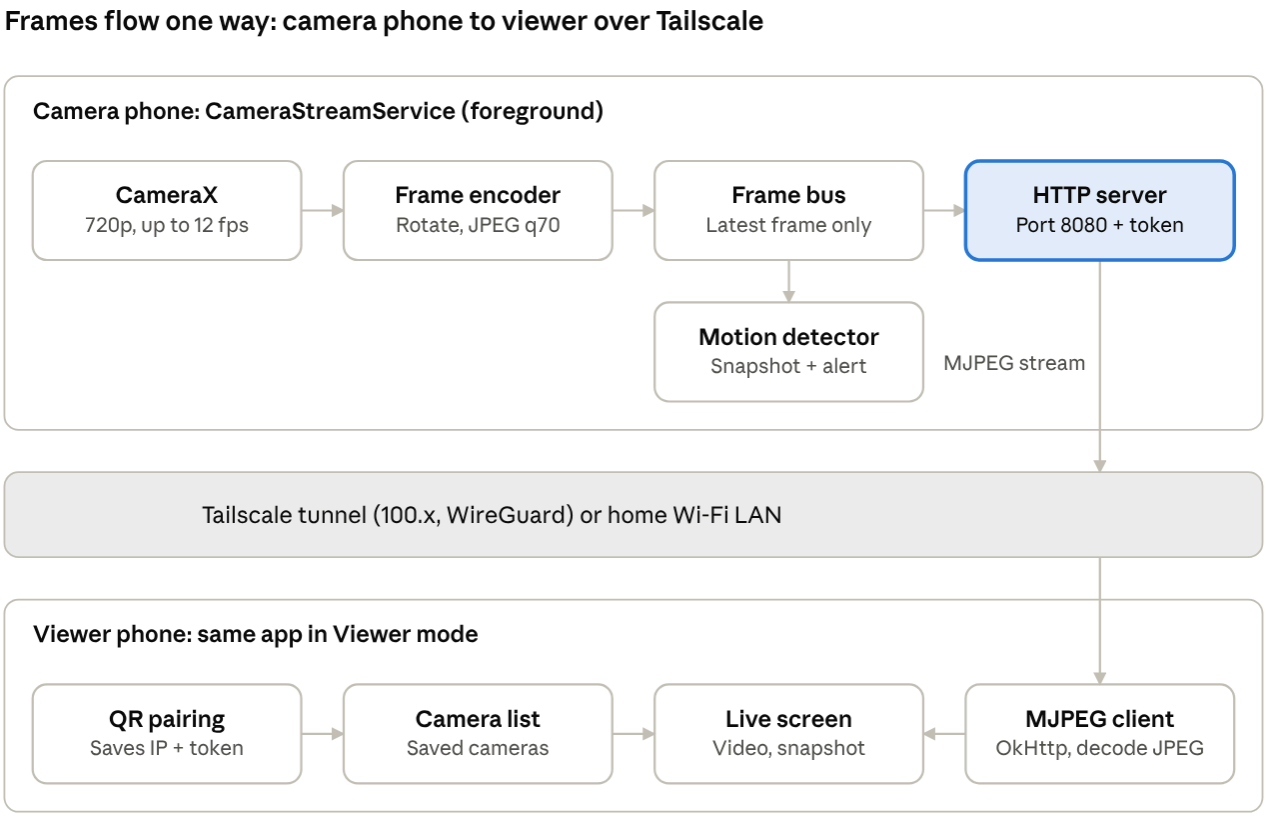
Goals for v1

- Live view from anywhere through Tailscale, with no port forwarding (works behind Jio AirFiber CGNAT).
- Password protection on by default, unlike IP Webcam.
- Runs 24x7 with the screen off, on charger, without Android killing it.
- Pair a viewer in seconds by scanning a QR code shown on the camera phone.
- Motion detection that saves snapshots and can raise an alert.

Not in v1: H.264 streaming, audio, cloud storage, multi-user accounts. These are in the Roadmap.

Tech choices: Kotlin, Jetpack Compose, CameraX, Ktor server (CIO engine) on the camera, OkHttp on the viewer, DataStore for settings. minSdk 26 (Android 8.0), targetSdk = latest stable.

System architecture



HomeCam architecture: camera phone, network, viewer phone

The HTTP server is the only part that faces the network; the viewer pulls frames from it through Tailscale (anywhere) or the home Wi-Fi (at home). Motion detection runs on the same frames, so it costs no extra camera work.

Project structure

Create one project in Android Studio with **New Project > Empty Activity (Compose)**, package `com.nitish.homecam`, single `app` module. Split code by mode so each half can be built and tested on its own.

```
com.nitish.homecam
├── MainActivity.kt          // mode picker: Camera or Viewer
├── core/
│   ├── Settings.kt          // DataStore: mode, token, quality, port
│   ├── AuthToken.kt         // generate + check the access token
│   └── NetUtils.kt          // find LAN and Tailscale (100.x) addresses
├── camera/
│   ├── CameraStreamService.kt  // foreground service, owns camera + server
│   ├── FrameEncoder.kt      // ImageProxy -> rotated JPEG bytes
│   ├── FrameBus.kt          // latest JPEG frame, shared by all clients
```

```
|   ├── MotionDetector.kt    // frame difference -> motion events
|   ├── MjpegServer.kt       // Ktor routes: /stream.mjpg, /snapshot.jpg,
/status
|   └── CameraScreen.kt      // start/stop, addresses, QR code
├── viewer/
|   ├── CameraRepository.kt  // saved cameras (name, url, token)
|   ├── MjpegClient.kt       // OkHttp stream -> Bitmap frames
|   ├── CameraListScreen.kt
|   └── LiveScreen.kt
└── pairing/
    ├── QrCode.kt            // make QR on camera side
    └── QrScanner.kt         // scan QR on viewer side
```

Dependencies (add through the version catalog; pick the latest stable version of each):

Purpose	Library
Camera capture	`androidx.camera:camera-core`, `camera-camera2`, `camera-lifecycle`
Service with lifecycle (CameraX binds to it)	`androidx.lifecycle:lifecycle-service`
HTTP server on camera phone	`io.ktor:ktor-server-core`, `io.ktor:ktor-server-cio`
HTTP client on viewer	`com.squareup.okhttp3:okhttp`
Settings storage	`androidx.datastore:datastore-preferences`
QR make + scan	`com.journeyapps:zxing-android-embedded` (includes ZXing core)
UI	Jetpack Compose BOM, Material 3, Navigation Compose

Permissions (`AndroidManifest.xml`):

Permission	Why
`CAMERA`	Capture video (runtime permission)
`INTERNET`, `ACCESS_NETWORK_STATE`, `ACCESS_WIFI_STATE`	Serve and read the stream, list IP addresses

Permission	Why
`FOREGROUND_SERVICE`, `FOREGROUND_SERVICE_CAMERA`	Keep the camera running in the background (Android 14+ needs the typed one)
`WAKE_LOCK`	Keep the CPU awake with the screen off
`POST_NOTIFICATIONS`	Service notification and motion alerts (runtime on Android 13+)
`REQUEST_IGNORE_BATTERY_OPTIMIZATIONS`	Ask to be exempt from battery saver

Camera mode (caster)

The camera phone runs everything inside one foreground service, so it keeps working with the screen off and the app closed.

Capture. Bind only a CameraX `ImageAnalysis` use case (no `Preview`) to the service's lifecycle. Without a preview the screen can stay off. Use `STRATEGY_KEEP_ONLY_LATEST` so slow encoding drops frames instead of queuing them, and `OUTPUT_IMAGE_FORMAT_RGBA_8888` so `ImageProxy.toBitmap()` works directly.

Encode. For each frame: `toBitmap()`, rotate by `imageInfo.rotationDegrees`, then `Bitmap.compress(JPEG, quality)`. Defaults: 1280x720, quality 70, max 12 fps (skip frames that arrive sooner than 1000/fps ms). On a weak phone this is the main CPU cost; lower resolution first if it heats up.

Share. `FrameBus` holds a `MutableStateFlow<ByteArray?>` with the latest JPEG. Each connected viewer collects it on its own coroutine with `conflate()`, so a slow viewer on mobile data skips frames and never slows the others.

Serve. A Ktor CIO server on port 8080 (configurable) exposes the stream, a snapshot and a status page (see Network protocol). The server starts and stops with the service.

Motion detection. Every 5th frame, scale the bitmap to 64x48, convert to grey, and compare it with the previous small frame. If the share of pixels whose brightness changed by more than 25 is above a threshold (default 2%, set by a sensitivity slider), raise a motion event. On motion: save the full JPEG to app storage with a timestamp, post a local notification, and ignore further events for 30 s (cooldown). Thresholds are starting values to tune on your phone.

Camera screen (UI).

- Start / Stop button for the service.

- Addresses: LAN (`192.168.31.x`) and Tailscale (`100.x`) with port, found by `NetUtils` .
- QR code for pairing (see Network protocol).
- Settings: resolution, fps, quality, front/back lens, motion sensitivity, regenerate token.
- Live counters: viewers connected, current fps, last motion time.

Viewer mode

The viewer is a normal foreground app: a list of saved cameras, and a live screen for one camera.

Camera list screen.

- Each saved camera: name, address, online/offline dot (from a `/status` call every 10 s while the screen is open).
- Add camera: **Scan QR** (fills everything) or **Enter manually** (name, `http://100.x.x.x:8080` , token).
- Long-press to rename or delete. Stored in DataStore as a small JSON list.

Live screen.

- `MjpegClient` opens `/stream.mjpg` with OkHttp and turns each JPEG part into a `Bitmap` ; Compose shows the latest one in an `Image` (fit to width, pinch-zoom optional).
- On disconnect, retry every 2 s and show "Reconnecting…" over the last frame.
- Buttons: **Snapshot** (save current frame to Gallery via MediaStore), **Full screen / landscape**, **Torch** and **Switch lens** (call the camera's control endpoints).
- Small overlay: fps received, connection type (Tailscale or LAN), last motion time.

Battery on the viewer. The stream runs only while the live screen is visible; leaving the screen cancels the coroutine and closes the HTTP connection.

Network protocol

Plain HTTP on port 8080, every request authenticated with a token. Tailscale already encrypts the link (WireGuard), so HTTPS is not needed for v1.

Method + path	Returns	Notes
`GET /stream.mjpg`	`multipart/x-mixed-replace; boundary=frame`	Endless JPEG parts; opens in Chrome too

Method + path	Returns	Notes
`GET /snapshot.jpg`	One `image/jpeg`	Latest frame
`GET /status`	JSON: name, fps, viewers, lastMotion, battery %, temperature	Used for the online dot
`POST /control/torch?on=1`	204	Flash on/off
`POST /control/lens?facing=back`	204	`back` or `front`
`GET /motion`	JSON list of saved motion snapshots	v1.1

MJPEG part format (repeated for every frame):

```
--frame\r\n
Content-Type: image/jpeg\r\n
Content-Length: 48213\r\n
\r\n
<JPEG bytes>\r\n
```

Authentication. On first start the camera generates a random 32-byte token (Base64URL) with `SecureRandom` and keeps it in DataStore. The viewer sends `Authorization: Bearer <token>`. For opening in a browser, `?t=<token>` in the URL is also accepted. Wrong or missing token → 401, and after 10 failures from one address in a minute, that address is blocked for 10 minutes.

Pairing by QR. The camera screen shows a QR code holding:

```
homecam://pair?name=Gate&host=100.126.131.35&port=8080&t=<token>
```

The viewer scans it, saves the camera, and opens the live screen. Regenerating the token on the camera unpairs every viewer.

Security

The stream is protected by default; nothing can be watched without the token, even on the home Wi-Fi.

- **Token always on.** There is no "no password" option. This fixes what you saw with IP Webcam opening without a login.

- **IPv4 only.** Bind the server to `0.0.0.0`, which does not listen on IPv6. Your AirFiber gives every device a public `2409:…` IPv6 address, so an IPv6 listener could be reachable from the internet.
- **Allow-list of source addresses.** Accept connections only from `100.64.0.0/10` (Tailscale) and private LAN ranges (`192.168.0.0/16`, `10.0.0.0/8`, `172.16.0.0/12`); reject everything else before the token check. Add a setting **Tailscale only** that drops LAN access too.
- **Token handling.** Compare tokens with `MessageDigest.isEqual` (constant time). Never write the token to logs. Show it only inside the QR code and behind a "Show token" tap.
- **Plain HTTP on LAN.** Over Tailscale the link is encrypted; on the LAN it is not, so anyone on your Wi-Fi could sniff the token. If that matters, use **Tailscale only** mode.
- **Snapshots stay private.** Save motion snapshots in app-private storage (`filesDir`), not the shared Gallery, unless the user taps Save.

Background and power

The camera must survive days with the screen off; Android's power management is the main thing that breaks this.

- **Foreground service, type camera.** `CameraStreamService` extends `LifecycleService` and calls `ServiceCompat.startForeground(..., FOREGROUND_SERVICE_TYPE_CAMERA)` with an ongoing notification ("HomeCam is streaming · 2 viewers" + Stop action).
- **Start it from the screen.** Android 11+ does not let a service open the camera if it was started from the background. Always start it from the Camera screen with the app open. Do not try to auto-start camera on boot; after a reboot, show a notification asking the user to tap to resume.
- **Wake locks.** Hold a `PARTIAL_WAKE_LOCK` (CPU) and a `WifiManager.WifiLock` (keeps Wi-Fi at full speed) while streaming; release both in `onDestroy`.
- **Battery optimisation.** On first run, ask for exemption (`ACTION_REQUEST_IGNORE_BATTERY_OPTIMIZATIONS`) and show the user how to set **Battery: Unrestricted** for HomeCam and Tailscale. Realme/ColorOS also has an auto-launch and background-freeze setting worth turning off.
- **Heat and battery health.** Default to 720p at 12 fps. Read battery temperature from `ACTION_BATTERY_CHANGED`; above 42 °C drop to 480p and 5 fps, and show a warning in `/status`. Advise a charge limit (85%) if the phone offers one, since a phone on charge 24x7 can swell.

- **Self-healing.** If the camera closes with an error, wait 5 s and rebind. If Tailscale's address disappears, keep serving LAN and recheck addresses every 30 s.

Development plan

Build in seven phases; each ends with something you can test on the two phones. Durations are rough estimates for evening work.

1. **Setup (half a day).** New Compose project, add dependencies and permissions, `MainActivity` with two buttons (Camera / Viewer) that open empty screens. Enable USB debugging on both phones.
   - Done when: the app installs on both phones and both buttons open their screens.
2. **Capture + encode (1 day).** Camera screen asks for camera permission, binds `ImageAnalysis` to the Activity for now, encodes JPEG, and shows the latest frame as an `Image` with an fps counter.
   - Done when: you see your own camera image at 10+ fps from encoded JPEGs.
3. **MJPEG server (1 day).** Add `FrameBus` and `MjpegServer`; show the LAN address on screen.
   - Done when: `http://192.168.31.x:8080/stream.mjpg` plays in Chrome on the S24 FE.
4. **Foreground service (1 day).** Move camera + server into `CameraStreamService`, add notification, wake locks, battery-optimisation prompt.
   - Done when: the stream keeps playing for 1 hour with the old phone's screen off.
5. **Security + pairing (1 day).** Token, allow-list, IPv4 binding, rate limit, QR code on camera screen.
   - Done when: Chrome without `?t=` gets 401, and through Tailscale `http://100.x:8080/stream.mjpg?t=…` plays on mobile data.
6. **Viewer mode (2 days).** Camera list, QR scan, `MjpegClient`, Live screen, snapshot, reconnect.
   - Done when: scan QR → live video on mobile data, and it recovers after you toggle the old phone's Wi-Fi.
7. **Motion + polish (2 days).** `MotionDetector`, saved snapshots, local notification, `/status`, heat protection, settings.
   - Done when: walking past the camera saves a snapshot and the viewer shows the new last-motion time.

Install on the old phone with **Run** over USB, or build a debug APK (**Build > Build APK(s)**) and copy it over.

Key code

These are working starting points for the core pieces (Kotlin, CameraX 1.3+, Ktor 3, OkHttp 4); imports and the notification channel are left out for space.

Manifest: the service entry

```
<uses-feature android:name="android.hardware.camera" android:required="false"
/>

<service
    android:name=".camera.CameraStreamService"
    android:foregroundServiceType="camera"
    android:exported="false" />
```

FrameBus.kt

```
object FrameBus {
    val latest = MutableStateFlow<ByteArray?>(null)   // latest JPEG
}
```

FrameEncoder.kt: upright bitmap + JPEG, with an fps cap

```
class FrameEncoder(private val quality: Int = 70, maxFps: Int = 12) {
    private val minGapMs = 1000L / maxFps
    private var lastMs = 0L

    fun toUprightBitmap(image: ImageProxy): Bitmap? {
        val now = SystemClock.elapsedRealtime()
        if (now - lastMs < minGapMs) return null
        lastMs = now
        val src = image.toBitmap()
        val deg = image.imageInfo.rotationDegrees
        if (deg == 0) return src
        val m = Matrix().apply { postRotate(deg.toFloat()) }
        return Bitmap.createBitmap(src, 0, 0, src.width, src.height, m, true)
    }

    fun toJpeg(bmp: Bitmap): ByteArray = ByteArrayOutputStream().use { out ->
        bmp.compress(Bitmap.CompressFormat.JPEG, quality, out)
        out.toByteArray()
    }
}
```

CameraStreamService.kt: camera + server in a foreground service

```
class CameraStreamService : LifecycleService() {
    private val executor = Executors.newSingleThreadExecutor()
    private val encoder = FrameEncoder()
    private val motion = MotionDetector { /* save JPEG + post notification */
}
    private var server: MjpegServer? = null
    private var wakeLock: PowerManager.WakeLock? = null

    override fun onStartCommand(intent: Intent?, flags: Int, startId: Int):
Int {
        super.onStartCommand(intent, flags, startId)
        val token = intent?.getStringExtra("token") ?: return
START_NOT_STICKY
        ServiceCompat.startForeground(
            this, 1, buildNotification(),
            if (Build.VERSION.SDK_INT >= 30)
ServiceInfo.FOREGROUND_SERVICE_TYPE_CAMERA else 0
        )
        wakeLock = getSystemService(PowerManager::class.java)
            .newWakeLock(PowerManager.PARTIAL_WAKE_LOCK, "HomeCam:stream")
            .apply { acquire() }
        startCamera()
        server = MjpegServer(port = 8080, token = token).also { it.start() }
        return START_NOT_STICKY   // camera can't restart from background
anyway
    }

    private fun startCamera() {
        val future = ProcessCameraProvider.getInstance(this)
        future.addListener({
            val provider = future.get()
            val analysis = ImageAnalysis.Builder()
                .setResolutionSelector(
                    ResolutionSelector.Builder().setResolutionStrategy(
                        ResolutionStrategy(Size(1280, 720),

ResolutionStrategy.FALLBACK_RULE_CLOSEST_HIGHER_THEN_LOWER)
                    ).build()
                )

.setBackpressureStrategy(ImageAnalysis.STRATEGY_KEEP_ONLY_LATEST)

.setOutputImageFormat(ImageAnalysis.OUTPUT_IMAGE_FORMAT_RGBA_8888)
                .build()
            analysis.setAnalyzer(executor) { image ->
                image.use {
```

```
                    val bmp = encoder.toUprightBitmap(it) ?: return@use
                    FrameBus.latest.value = encoder.toJpeg(bmp)
                    motion.process(bmp)
                }
            }
            provider.unbindAll()
            provider.bindToLifecycle(this,
CameraSelector.DEFAULT_BACK_CAMERA, analysis)
        }, ContextCompat.getMainExecutor(this))
    }

    override fun onDestroy() {
        server?.stop()
        wakeLock?.release()
        executor.shutdown()
        super.onDestroy()
    }
}
```

Start it from the Camera screen with `ContextCompat.startForegroundService(context, Intent(context, CameraStreamService::class.java).putExtra("token", token))`.

MjpegServer.kt: Ktor routes with token and source check

```
class MjpegServer(private val port: Int, private val token: String) {
    private var server: EmbeddedServer<*, *>? = null

    fun start() {
        server = embeddedServer(CIO, port = port, host = "0.0.0.0") {   //
IPv4 only
            routing {
                get("/stream.mjpg") {
                    if (!authorized(call)) return@get
call.respond(HttpStatusCode.Unauthorized)
                    val type = ContentType("multipart", "x-mixed-replace")
                        .withParameter("boundary", "frame")
                    call.respondBytesWriter(contentType = type) {
                        FrameBus.latest.filterNotNull().conflate().collect {
jpeg ->
                            writeStringUtf8("--frame\r\nContent-Type:
image/jpeg\r\n" +
                                "Content-Length: ${jpeg.size}\r\n\r\n")
                            writeFully(jpeg)
                            writeStringUtf8("\r\n")
                            flush()
                        }
```

```
                    }
                }
                get("/snapshot.jpg") {
                    if (!authorized(call)) return@get
call.respond(HttpStatusCode.Unauthorized)
                    val jpeg = FrameBus.latest.value
                        ?: return@get
call.respond(HttpStatusCode.ServiceUnavailable)
                    call.respondBytes(jpeg, ContentType.Image.JPEG)
                }
            }
        }.start(wait = false)
    }

    fun stop() { server?.stop(500, 1000) }

    private fun authorized(call: ApplicationCall): Boolean {
        if (!NetUtils.isAllowedSource(call.request.origin.remoteAddress))
return false
        val given =
call.request.headers["Authorization"]?.removePrefix("Bearer ")
            ?: call.request.queryParameters["t"] ?: return false
        return MessageDigest.isEqual(given.toByteArray(),
token.toByteArray())
    }
}
```

NetUtils.kt + AuthToken.kt

```
object NetUtils {
    fun isAllowedSource(ip: String): Boolean {
        val p = ip.split('.').mapNotNull { it.toIntOrNull() }
        if (p.size != 4) return false                       // rejects IPv6
        return p[0] == 10 || p[0] == 127 ||
            (p[0] == 192 && p[1] == 168) ||
            (p[0] == 172 && p[1] in 16..31) ||
            (p[0] == 100 && p[1] in 64..127)                 // Tailscale
    }
}

fun newToken(): String {
    val bytes = ByteArray(32).also { SecureRandom().nextBytes(it) }
    return Base64.encodeToString(bytes, Base64.URL_SAFE or Base64.NO_WRAP or
Base64.NO_PADDING)
}
```

MotionDetector.kt

```
class MotionDetector(
    private val threshold: Float = 0.02f,     // 2% of pixels changed
    private val onMotion: () -> Unit
) {
    private var prev: IntArray? = null
    private var count = 0
    private var lastEventMs = 0L

    fun process(frame: Bitmap) {
        if (++count % 5 != 0) return
        val small = Bitmap.createScaledBitmap(frame, 64, 48, true)
        val px = IntArray(64 * 48).also { small.getPixels(it, 0, 64, 0, 0,
64, 48) }
        val grey = IntArray(px.size) { i ->
            val c = px[i]
            ((c shr 16 and 0xFF) * 30 + (c shr 8 and 0xFF) * 59 + (c and
0xFF) * 11) / 100
        }
        val old = prev
        prev = grey
        if (old == null) return
        val changed = grey.indices.count { abs(grey[it] - old[it]) > 25 }
        val now = SystemClock.elapsedRealtime()
        if (changed.toFloat() / grey.size > threshold && now - lastEventMs >
30_000) {
            lastEventMs = now
            onMotion()
        }
    }
}
```

MjpegClient.kt (viewer): OkHttp stream to Bitmaps

```
private val client = OkHttpClient.Builder()
    .readTimeout(10, TimeUnit.SECONDS)        // no frame for 10 s = dropped
    .build()

fun mjpegFrames(url: String, token: String): Flow<Bitmap> = flow {
    val req = Request.Builder().url(url).header("Authorization", "Bearer
$token").build()
    client.newCall(req).execute().use { resp ->
        if (!resp.isSuccessful) throw IOException("HTTP ${resp.code}")
        val src = resp.body!!.source()
        while (true) {
```

```
            var len = -1
            while (true) {                          // read part headers
                val line = src.readUtf8LineStrict()
                if (line.isEmpty()) { if (len >= 0) break else continue }
                if (line.startsWith("Content-Length", ignoreCase = true))
                    len = line.substringAfter(':').trim().toInt()
            }
            val bytes = src.readByteArray(len.toLong())
            BitmapFactory.decodeByteArray(bytes, 0, bytes.size)?.let {
emit(it) }
        }
    }
}.flowOn(Dispatchers.IO)
```

LiveScreen.kt (viewer): show frames, reconnect on error

```
@Composable
fun LiveView(url: String, token: String) {
    val frame by produceState<Bitmap?>(null, url) {
        mjpegFrames("$url/stream.mjpg", token)
            .retryWhen { _, _ -> delay(2000); true }
            .collect { value = it }
    }
    frame?.let {
        Image(it.asImageBitmap(), contentDescription = "Live camera",
            modifier = Modifier.fillMaxWidth(), contentScale =
ContentScale.Fit)
    } ?: Text("Connecting…")
}
```

Testing checklist

- [ ] Stream plays in Chrome on the same Wi-Fi with `?t=<token>`
- [ ] Without a token, every endpoint returns 401
- [ ] IPv6 address (`http://[2409:…]:8080`) does not connect
- [ ] Through Tailscale on mobile data, video plays at 8+ fps
- [ ] Two viewers at once; one on a slow network does not slow the other
- [ ] Old phone screen off for 8 hours: stream still up, notification visible
- [ ] Toggle the old phone's Wi-Fi off and on: viewer reconnects on its own
- [ ] Walk past the camera: snapshot saved, notification shown, then 30 s cooldown
- [ ] Battery temperature stays under 42 °C after 2 hours on charge
- [ ] Regenerate token: old viewers get 401, re-scan QR fixes it

Roadmap

After v1 works, add features in this order; each builds on the last.

1. **Motion alerts on the viewer phone.** Push to the S24 FE through a free service such as ntfy or a Telegram bot, with the snapshot attached.
2. **Clip recording.** On motion, record 15–30 s of H.264 with `MediaCodec` + `MediaMuxer` into app storage; auto-delete clips older than 7 days.
3. **H.264 live stream.** Replace MJPEG with H.264 over RTSP or WebRTC: roughly a tenth of the data at the same quality, and compatible with Frigate or tinyCam.
4. **Person detection.** Run a small YOLO model with LiteRT (TensorFlow Lite) on the motion frames; alert only when a person is found, which removes alarms from trees, pets and light changes.
5. **Two-way audio and multi-camera grid.** Microphone stream, talk-back, and a 2x2 grid of cameras in Viewer mode.